

1. Lineare Optimierung

Beschreibung

Lineare Optimierung ist ein Teilgebiet des Operations Research (OR). Reale Probleme, bspw. aus dem Bereich der Gesamtwirtschaft, einer Unternehmung oder der Versorgung des Militärs werden durch mathematische Modelle beschrieben und mit mathematischen Methoden im Anschluss gelöst. Die so gefundenen Lösungen dienen der Vorbereitung von Entscheidungen die bezüglich der Zielsetzungen optimiert werden sollen.

Definition und Darstellung von linearen Programmen

Beispiel

Zur Fertigung zweier Produkte P_1 und P_2 werden 4 Maschinen A, B, C und D benötigt. Die zur Herstellung für eine Einheit des Produktes P_1 oder P_2 benötigten Maschinenzeiten (h) sind in der nachfolgenden Tabelle zusammengefasst. Um eine Einheit des Produktes P_1 herzustellen, benötigt man 2 h (Stunden) Bearbeitungszeit auf Maschine A, 2 h auf Maschine B sowie 2 h auf Maschine D. Auf der Maschine A benötigt das erste Produkt 2 h zur Bearbeitung, Produkt 2 benötigt 3 h. Die Maschine A hat bspw. eine begrenzte Kapazität von 180 h, welche nicht überschritten werden kann.

Maschine	P_1	P_2	Kapazität (h)
A	2	3	180
B	2	1,5	150
C	0	3	120
D	2	0	190

Das Produkt P_1 erziele einen Stückgewinn von 200€, das Produkt P_2 einen Stückgewinn von 500€.

Bestimmen Sie die gewinnmaximale Produktion.

Mathematisches Modell

Zielfunktion:

Maximiere $G =$

unter den Restriktionen:

- | | |
|----|------------|
| 1) | Maschine A |
| 2) | Maschine B |
| 3) | Maschine C |
| 4) | Maschine D |

mit den Nichtnegativitätsbedingungen

- | | |
|----|--------------|
| 5) | $x_1 \geq 0$ |
| 6) | $x_2 \geq 0$ |

Zeichnerische Lösung für zwei Unbekannte

Jeder Punkt im zulässigen (markierten Bereich) liefert ein produktionstechnisch mögliches Produktionsprogramm.

Um die Optimallösung zu ermitteln trägt man eine Isogewinngerade in das Koordinatensystem ein und ermittelt durch Verschiebung derselben die Optimallösung.

Optimallösung: $(x_1, x_2) = (\quad \text{ME}, \quad \text{ME})$,

Gewinnmaximum $G =$

Möglichkeit der semigraphische Lösung

Definition „Lineares Programm“ (LP)

Maximiere Zielfunktion $C_1 \cdot x_1 + C_2 \cdot x_2 + \dots + C_n \cdot x_n$

unter den Restriktionen

$$a_{i1} \cdot x_1 + a_{i2} \cdot x_2 + \dots + a_{in} \cdot x_n \leq b_i \quad \text{für } i = 1, \dots, m$$

und den Nichtnegativitätsbedingungen $x_1, \dots, x_n \geq 0$.

Weitere Schreibweise:

Für das Eingangsbeispiel:

Weitere Bezeichnungen:

$$M = \{(x_1, \dots, x_n) : a_{i1} \cdot x_1 + a_{i2} \cdot x_2 + \dots + a_{in} \cdot x_n \leq b_i \quad \text{für } i = 1, \dots, m \text{ und } x_1, \dots, x_n \geq 0\}$$

heißt die **Menge der zulässigen Punkte** und wird, falls M beschränkt ist auch mit

Lösungspolyeder bezeichnet.

$x^* \in M$ heißt **Optimallösung**, falls

$$\max_{x \in M} (C_1 \cdot x_1 + C_2 \cdot x_2 + \dots + C_n \cdot x_n) = C_1 \cdot x^*_1 + C_2 \cdot x^*_2 + \dots + C_n \cdot x^*_n < \infty$$

Eine Ecke ist der Schnittpunkt von mindestens zwei Restriktionen die die Menge der zulässigen Punkte beschränken.

Simplex-Algorithmus

Die Optimallösung eines LP wird immer auch an einer Ecke angenommen¹. Ausgehend vom Ursprung erfolgt in jeder Iteration ein Eckentausch (Basistausch). Mit jedem Eckentausch wird der Zielfunktionswert verbessert. Da der Lösungsraum endlich viele Ecken besitzt, wird die Optimallösung auch nach endlich vielen Schritten erzielt.

Anwendung des Simplex-Algorithmus auf das Eingangsbeispiel

x_1	x_2	y_1	y_2	y_3	y_4	b
		0	0	0	0	0
2	3					180
2	1,5					150
0	3 *					120
2	0					190

x_1	x_2	y_1	y_2	y_3	y_4	b

x_1	x_2	y_1	y_2	y_3	y_4	b

¹ Eckentheorem: Ist der zulässige Bereich eines linearen Optimierungsproblems beschränkt (konvexes Polyeder), so existiert das Maximum oder Minimum und wird in mindestens einem Eckpunkt angenommen. Ist der zulässige Bereich unbeschränkt und existiert das Maximum oder Minimum, so wird es ebenfalls in mindestens einem Eckpunkt angenommen.

Ökonomische Interpretation des Endtableaus

Die optimale Basislösung lautet:

Der maximale Gewinn beträgt:

$$\begin{pmatrix} x_1 \\ x_2 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} = \begin{pmatrix} 30 \\ 40 \\ 0 \\ 30 \\ 0 \\ 130 \end{pmatrix}$$

$$G_{\max} = 26.000 \text{ €}$$

$$x_1 = 30 \quad \rightarrow$$

$$x_2 = 40 \quad \rightarrow$$

$$y_1 = 0 \quad \rightarrow$$

$$y_2 = 30 \quad \rightarrow$$

$$y_3 = 0 \quad \rightarrow$$

$$y_4 = 130 \quad \rightarrow$$

Schattenpreise:

x_1	x_2	y_1	y_2	y_3	y_4	G	b
0	0	100	0	$66\frac{2}{3}$	0	1	26.000

$$\rightarrow 100 y_1 + 66\frac{2}{3} y_3 + G = 26.000$$

$$\leftrightarrow G = 26.000 - 100 y_1 - 66\frac{2}{3} y_3$$

Eckentausch im Simplextableau

Sei a_{rs} das Pivotelement.

1. **Umformung:** $a'_{rk} = a_{rk} \cdot \frac{1}{a_{rs}}$ (Division der Pivotzeile durch das Pivotelement)
2. **Umformung:** $a'_{is} = 0$ (Alle Elemente der Pivotspalte, mit Ausnahme des Pivotelements werden Null gesetzt)
3. **Umformung:** $a'_{ik} = a_{ik} - \frac{a_{is} \cdot a_{rk}}{a_{rs}}$ (Rechteckregel für alle sonstigen Elemente)

Sonderfälle des Simplex-Algorithmus

Unendlich viele Lösungen

1. Beispiel

$$\begin{array}{ll} \text{maximiere:} & G = 2 \cdot x_1 + 3 \cdot x_2 \\ \text{unter den Restriktionen:} & \begin{array}{ll} \text{a} & 2 \cdot x_1 + 3 \cdot x_2 \leq 180 \\ \text{b} & 2 \cdot x_1 + 1.5 \cdot x_2 \leq 150 \\ \text{c} & 3 \cdot x_2 \leq 120 \\ \text{d} & 2 \cdot x_1 \leq 190 \end{array} \end{array} \quad x_1, x_2 \geq 0$$

2. Beispiel

Maximiere $3x_1 + 3x_2$

unter den Restriktionen $x_1 + x_2 \leq 4$

$$x_2 \leq 3, \quad x_1, x_2 \geq 0.$$

Bestimmen Sie die Lösungen mit Hilfe einer Zeichnung und mit dem Simplex-Algorithmus.

Unbeschränkter Lösungsraum

Beispiel:

Maximiere $G = 200 \text{ €} \cdot x_1 + 500 \text{ €} \cdot x_2$

unter der Restriktion $3 \cdot x_2 \leq 120$

Simplex Algorithmus für allgemeine Lineare Programme

1. Minimierungsproblem
2. Gleichungen als Restriktionen
3. Variablen ohne Nichtnegativitätsbedingungen
 - a. Negative Variable
 - b. Variable ohne Vorzeichenbeschränkung

Unzulässige Ausgangslösung

Das Optimierungsproblem hat keine zulässige Basislösung, solange auf der rechten Seite des Anfangstableaus mindestens ein $b_i < 0$ ist.

Um eine zulässige Ausgangslösung zu erhalten, wird eine Phase I dem eigentlichen Algorithmus vorgeschaltet:

Regeln:

Zeilenwahl:

Solange es auf der rechten Seite noch $b_i < 0$ gibt, wähle man unter diesen das kleinste aus. (Pivotzeile)

Spaltenwahl:

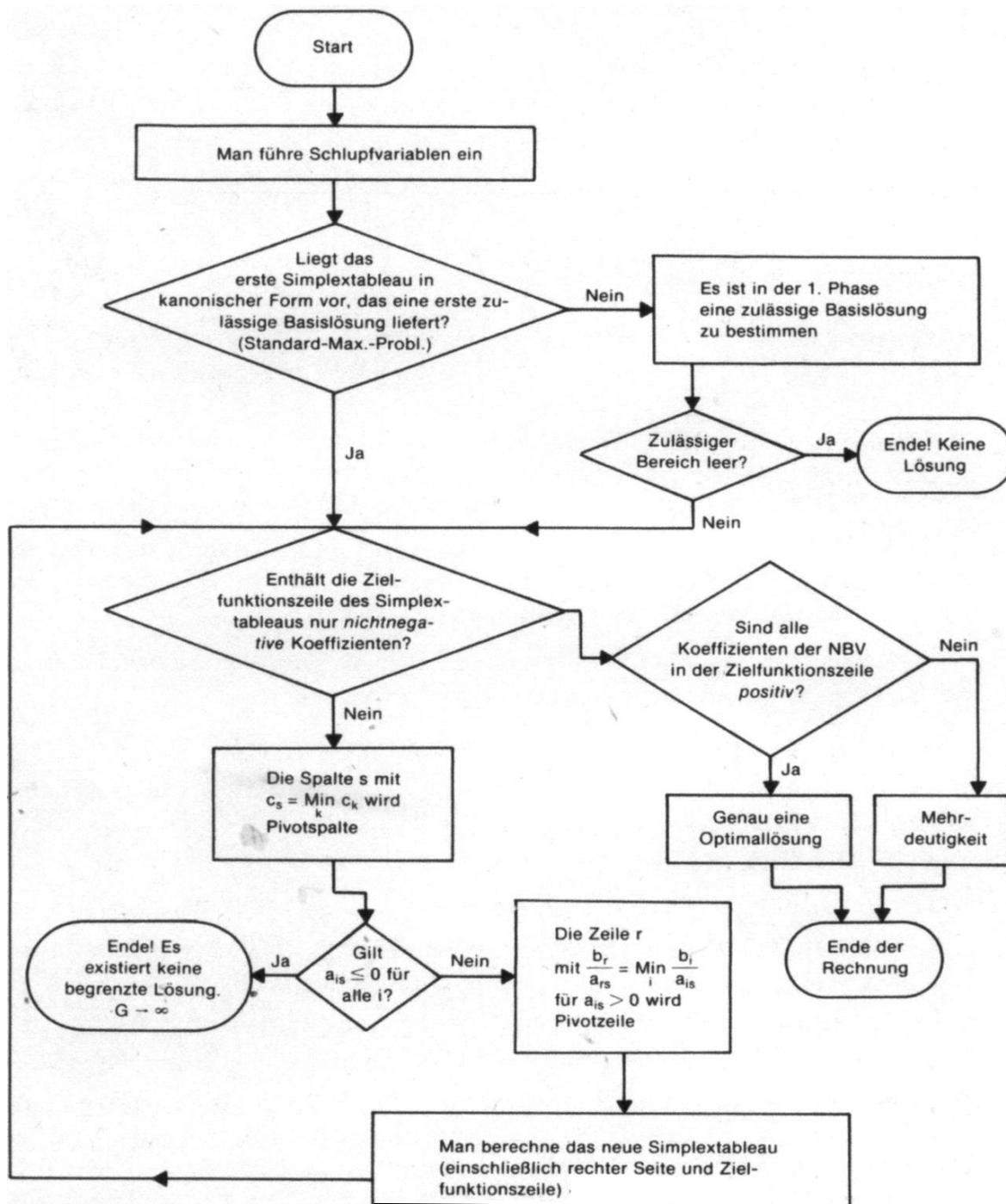
Sind in der gewählten Zeile alle Elemente größer oder gleich Null, so gibt es keine zulässige Lösung. STOP.

Sonst: Wähle unter den negativen Elementen das kleinste aus.

Beispiel

Im Ausgangsbeispiel wird die Lieferbedingung $x_1 + x_2 \geq 20$ hinzugefügt.

Simplex Algorithmus



Übungsaufgabe 1

Zur Fertigung zweier Produkte P_1 und P_2 müssen drei Maschinen A, B und C benutzt werden. Die zur Herstellung für eine Einheit benötigten Maschinenzeiten (in h), die Kapazitätsbeschränkungen und der Stückgewinn sind in der nachfolgenden Tabelle zusammengefasst:

	P_1	P_2	Kapazität
Maschine A	0	2	160
Maschine B	1	1	120
Maschine C	1	3	240
Gewinn/Stück in €	2	3	

Der Gewinn ist zu maximieren.

- Fertigen Sie eine Zeichnung und geben Sie die Optimallösung an.
- Angaben zur Lösung

Als Endtableau des Simplex- Algorithmus ergibt sich:

X_1	X_2	Y_1	Y_2	Y_3	RS	
0	0	0	1,5	0,5	300	Zielfunktionszeile
0	1	0	-0,5	0,5	60	Maschine A
0	0	1	1	-1	40	Maschine B
1	0	0	1,5	-0,5	60	Maschine C

Nehmen Sie Ihre Zeichnung und das Endtableau zu Hilfe und kreuzen Sie an:
(ME: Mengeneinheiten, GE: Geldeinheiten)

	Wahr	Falsch
Maschine A ist voll ausgelastet.		
Maschine B ist nicht voll ausgelastet.		
Erhöht man die Kapazität der Maschine C um 1h, so steigt der Gewinn um 0,5 GE.		
Fällt die Maschine B 1h aus, so fällt der Gewinn um 1,5 GE.		
Erhöht man die Kapazität von Maschine B um 1h, so steigt die Produktion von P_2 .		
Erhöht man die Kapazität von Maschine B um 1h, so steigt die Produktion von P_1 .		
Erhöht man die Kapazität der Maschine C, so fällt die Produktion von P_2 .		
Da Y_1 in der Basis ist, gibt es noch eine weitere Ecke.		
X_1 , X_2 und Y_1 sind Strukturvariablen		
Wäre der Stückgewinn beider Produkte 2 GE, so wäre $X_1 = 40$ ME und $X_2 = 80$ ME eine Optimallösung.		

Übungsaufgabe 2

Gegeben ist folgendes lineares (Produktions-) Programm:

Maximiere $z = 40x_1 + 60x_2$ unter den Restriktionen

- I. $1x_1 + 2x_2 \leq 32$ (Maschine A)
- II. $5x_1 + 6x_2 \leq 120$ (Maschine B)
- III. $x_1 \geq 4$ (Lieferbedingung)
- IV. $x_2 \geq 12$ (Lieferbedingung)

und den Nichtnegativitätsbedingungen: $x_i \geq 0$ für $i = 1, 2$

- a) Bestimmen Sie graphisch den zulässigen Lösungsraum. Kennzeichnen Sie diesen Lösungsraum in eindeutiger Weise.
- b) Ermitteln Sie graphisch die Optimallösung.
- c) Überprüfen Sie diese Lösung rechnerisch indem Sie die Ecken des Lösungspolyeders überprüfen.
- d) Nehmen wir an, Sie hätten die Möglichkeit eine der vier Restriktionen zu verändern. Welche Restriktion würden Sie (in welcher Weise) variieren um einen höheren Zielfunktionswert zu erreichen? Geben Sie die Grenze der Variation an.
- e) Geben Sie das Anfangstableau für den Simplex-Algorithmus an.
- f) Welche der Variablen werden nach Anwendung des Simplex-Algorithmus im Endtableau Basisvariable und welche Nichtbasisvariable sein? Geben Sie eine Begründung mithilfe Ihrer Zeichnung. (Es ist keine Rechnung erforderlich).

Dualität**Eingangsbeispiel:**

Ein zweites Unternehmen möchte die vier Maschinen des ersten Unternehmens mieten. Das erste Unternehmen wägt die Preise pro Maschinenstunde $w_1, \dots, w_4$ ab.

Ab welcher Höhe lohnt sich ein Angebot für das erste Unternehmen?

Formulieren Sie das Angebot aus der Sicht des zweiten Unternehmens. Welche Preise/Maschinenstunde wird das zweite Unternehmen anbieten?

Aufgabe: Berechnung von Schranken für das duale Problem

Definition

Zu jedem Linearen Programm gibt es ein anderes lineares Programm, das zum ursprünglichen primalen Programm als duales Programm bezeichnet wird.

Zum Standard-Maximierungsprogramm (PP)

Maximiere die Zielfunktion $C_1 \cdot x_1 + C_2 \cdot x_2 + \dots + C_n \cdot x_n$

unter den Restriktionen

$$a_{11} \cdot x_1 + a_{12} \cdot x_2 + \dots + a_{1n} \cdot x_n \leq b_1$$

.... ..

$$a_{i1} \cdot x_1 + a_{i2} \cdot x_2 + \dots + a_{in} \cdot x_n \leq b_i$$

.... ..

$$a_{m1} \cdot x_1 + a_{m2} \cdot x_2 + \dots + a_{mn} \cdot x_n \leq b_m$$

und den Nichtnegativitätsbedingungen $x_1, \dots, x_n \geq 0$

Einschließungssatz

- a) Seien x eine zulässige Lösung des PP und w eine zulässige Lösung des DP, dann gilt: $f_P(x) \leq f_D(w)$
- b) Wenn eines der zueinander dualen Programme eine optimale Lösung hat, so hat auch das andere Programm eine Optimallösung und die optimalen Zielfunktionswerte sind gleich.
Das heißt: Für optimale Lösungen x^* und w^* gilt: $f_P(x^*) = f_D(w^*)$.
- c) Aus a) und b) folgt der Einschließungssatz: $f_P(x) \leq f_P(x^*) = f_D(w^*) \leq f_D(w)$

Aus dem Einschließungssatz folgt:

Ist der Zielfunktionswert des PP (Maximierungsproblems) nach oben unbeschränkt, so hat das DP (Minimierungsproblem) keine zulässige Lösung.

Übungsaufgabe 1

PP: $\max \quad 3x_1 + 4x_2$

unter den Restriktionen $2x_1 + x_2 \leq 5$

$$x_1 + 3x_2 = 6 \quad \text{und } x_1, x_2 \geq 0$$

Formulieren Sie das dazu Duale Programm.

Übungsaufgabe 2

PP: $\max \quad 3x_1 + 4x_2$

unter den Restriktionen $2x_1 + x_2 \leq 5$

$$x_1 + 3x_2 \leq 6 \quad \text{und } x_1 \geq 0, x_2 \text{ vorzeichenunbeschränkt}$$

Formulieren Sie das dazu Duale Programm.

Sensitivitätsanalyse

Bei der Sensitivitätsanalyse wird untersucht, welche Auswirkung eine Änderung der Ausgangsdaten auf die gefundene Optimallösung hat. Es wird also die Stabilität der gefundenen optimalen Lösung überprüft. Dies ist für die Praxis von großer Bedeutung, da Preise, Lieferbedingungen, Deckungsbeiträge etc. meist Schwankungen unterworfen sind.

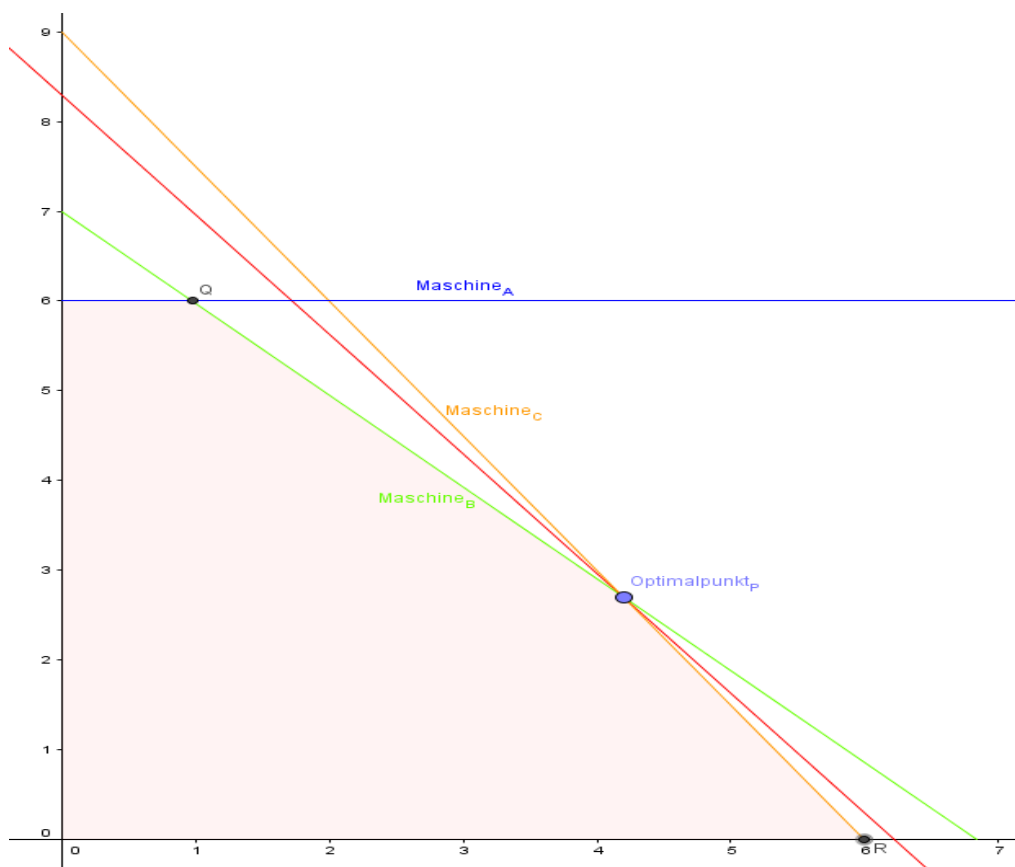
Beispiel

$$\max z = 4x_1 + 3x_2 \quad \text{unter}$$

$$x_2 \leq 6$$

$$x_1 + x_2 \leq 7$$

$$3x_1 + 2x_2 \leq 18, \quad x_1, x_2 \geq 0$$



Änderung der Zielfunktionsgeraden:

- Steigung Restriktionsgerade C < Steigung Zielfunktion < Steigung Restriktionsgerade B
→ Optimale Ecke bleibt erhalten
- Steigung Zielfunktion = Steigung Restriktionsgerade B
→ Jeder Punkt von PQ ist optimal
- Steigung Zielfunktion = Steigung Restriktionsgerade C
→ Jeder Punkt von PR ist optimal
- Steigung Zielfunktion > Steigung Restriktionsgerade B
→ Q ist die optimale Ecke
- Steigung Zielfunktion < Steigung Maschine C
→ R ist die optimale Ecke

Berechnung:**1. Berechnung der Steigungen der Zielfunktion und der aktiven Restriktionen:**

Zielfunktion: $c_1x_1 + c_2x_2 = Z$

Maschine B: $x_1 + x_2 = 7$

Maschine C: $3x_1 + 2x_2 = 18$

2. Optimalitätsbedingungen aufstellen:

Simultane Änderung der Zielfunktionskoeffizienten

Optimalitätsbedingung:

$$1 \leq \frac{4 + y_1}{3 + y_2} \leq \frac{3}{2}$$

Gewinnänderung:

$$\Delta z = y_1 x_1 + y_2 x_2$$

Änderung der aktiven Restriktionsgeraden

Die geringste Änderung einer aktiven Restriktionsgerade, verändert die optimale Ecke sofort. Die Änderung des Zielfunktionswertes wird durch den entsprechenden Schattenpreis der Schlupfvariable angegeben. Voraussetzung hierbei ist, dass sich lediglich die rechte Seite der Restriktion „geringfügig“ ändert. Ändert sich die rechte Seite der Restriktion um eine Einheit, so kann die Änderung im Endtableau des Simplex-Algorithmus abgelesen werden.

Beispiel: Änderung der Maschinenstunden von B

- Verringerung der Maschinenstunden B von 7 auf 6

Das ursprüngliche Optimaltableau lautet:

x_1	x_2	y_1	y_2	y_3	
0	0	0	1	1	25
0	0	1	-3	1	3
0	1	0	3	-1	3
1	0	0	-2	1	4

Simultane Änderung beider aktiver Restriktionen

- Änderung beider Maschinenstunden um β_1 bzw. β_2
 - Maschine B: $x_1 + x_2 = 7 + \beta_1$
 - Maschine C: $3x_1 + 2x_2 = 18 + \beta_2$

- Die Lösung des LGS führt zu folgenden Koordinaten des neuen Optimums P^*
 - $x_1 =$
 - $x_2 =$

- Zulässigkeitsbedingungen für P^*
 - $4 - 2\beta_1 + \beta_2 \geq 0$ (Nichtnegativitätsbedingung für x_1)
 - $3 + 3\beta_1 - \beta_2 \geq 0$ (Nichtnegativitätsbedingung für x_2)
 - x_1 und x_2 einsetzen in die nichtaktive Restriktion:

Beispiel:

Reduktion der Kapazität von Maschine B auf 5.5h und von Maschine C auf 14h

Änderung der inaktiven Restriktionsgerade

- Bei Verschiebung einer inaktiven Restriktionsgerade ändert sich weder das Optimum noch der optimale Zielfunktionswert, solange die optimale Ecke dazu nicht unzulässig wird.

Aufgabe

Führen Sie die Sensitivitätsanalyse für das Eingangsbeispiel im Skript durch

Zielfunktion: $Z = 200x_1 + 500x_2$

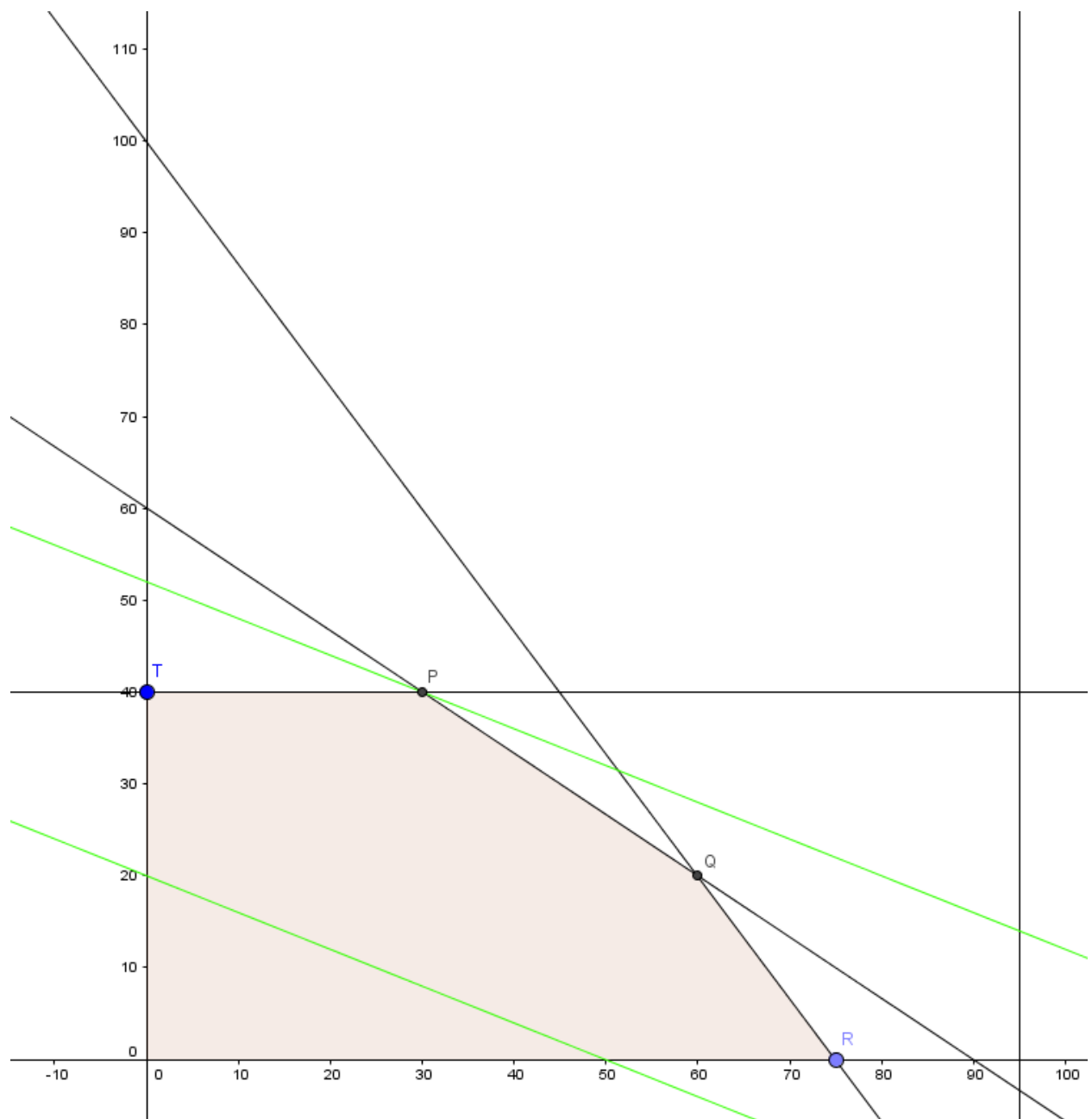
Restriktionen: Maschine A: $2x_1 + 3x_2 \leq 180$

Maschine B: $2x_1 + 1,5x_2 \leq 150$

Maschine C: $3x_2 \leq 120$

Maschine D: $2x_1 \leq 190$

Nichtnegativitätsbedingungen: $x_1 \geq 0, \quad x_2 \geq 0$



2. Ganzzahlige Optimierung (ILP)

Problemklassifikation

Optimierungsalgorithmen werden häufig wie Such- und Sortiervverfahren nach ihrer Komplexität klassifiziert. Die Zahl elementarer Rechenoperationen skaliert mit der Systemgröße n (Zahl der Orte beim TSP, Zahl der Gegenstände beim Rucksackproblem etc.). Lässt sich der Rechenaufwand $f(n)$ als Polynom schreiben, spricht man von P-Problemen. Ist $f(n)$ dagegen exponentiell oder enthält Ausdrücke wie $n!$, so spricht man von NP-harten oder NP-vollständigen Problemen. Auf die genauere Differenzierung gehen wir hier nicht ein. Folgende Tabelle zeigt eine Abschätzung der Rechenzeiten.

n	20	50	100

n	$2 \cdot 10^{-10} s$	$5 \cdot 10^{-10} s$	$10^{-9} s$
n^2	$4 \cdot 10^{-9} s$	$2,5 \cdot 10^{-8} s$	$10^{-7} s$
n^3	$8 \cdot 10^{-8} s$	$1,3 \cdot 10^{-6} s$	$10^{-5} s$
2^n	$10^{-5} s$	$0,13 d$	$4 \cdot 10^{11} a$
$n!$	$0,77 a$	$10^{46} a$	$3 \cdot 10^{139} a$

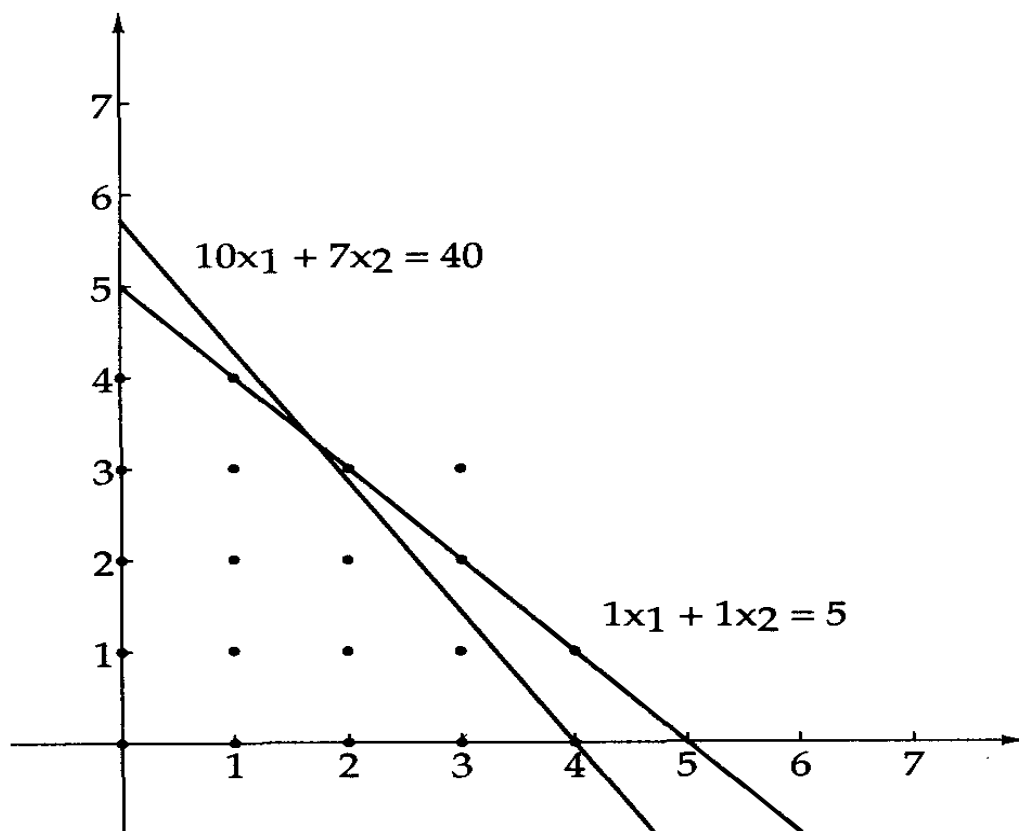
Vollständige Enumeration und Branch and Bound am Beispiel eines LP**Beispiel**

Maximiere $17x_1 + 12x_2$

unter

$$10x_1 + 7x_2 \leq 40$$

$$1x_1 + 1x_2 \leq 5 \quad x_i \geq 0 \text{ für } i = 1, 2 \text{ und ganzzahlig}$$



Branch and Bound Verfahren

„Branch & Bound“ heißt wörtlich übersetzt „Verzweigen - und – Abschätzen“, bzw. Schranken finden. Ein Problem wird in kleinere Einheiten (Teilprobleme) aufgeteilt. Jede Einheit wird für sich dahingehend untersucht, ob diese Einheit die optimale Lösung des Problems enthalten kann.

Die Teilung des Problems wird beim Branch & Bound Verfahren beispielsweise für die Lösung ganzzahliger Probleme dadurch vorgenommen, dass der zulässige Lösungsbereich systematisch zerstückelt wird. Man erhofft sich hiervon übersichtliche Lösungsmengen, die weiter untersucht werden. Der Zielfunktionswert wird durch die Lösung eines relaxierten Problems abgeschätzt. Hierbei wird die Bedingung der Ganzzahligkeit außer Kraft gesetzt. Man gewinnt damit also eine Abschätzung für das eigentliche ganzzahlige Problem, im Falle der Maximierung eine obere Schranke. Eine untere Schranke ist dann die bisher gefundene ganzzahlige Lösung mit dem höchsten Zielfunktionswert.

Gegeben ist hier beispielsweise ein Maximierungsprogramm.

Für die Teilprobleme gilt dann:

1. Enthält das relaxierte Problem (LP ohne Ganzzahligkeitsforderung) keine Lösung, so ist auch das ganzzahlige Problem unlösbar.
2. Ist die obere Schranke kleiner als eine bereits bekannte Lösung des ganzzahligen Problems, so kann in diesem Teilproblem keine bessere Lösung gefunden werden.
3. Ist schließlich die Lösung im Sinne des ganzzahligen Problems zulässig, so vergleicht man diese mit der besten bekannten zulässigen Lösung und ersetzt diese gegebenenfalls.

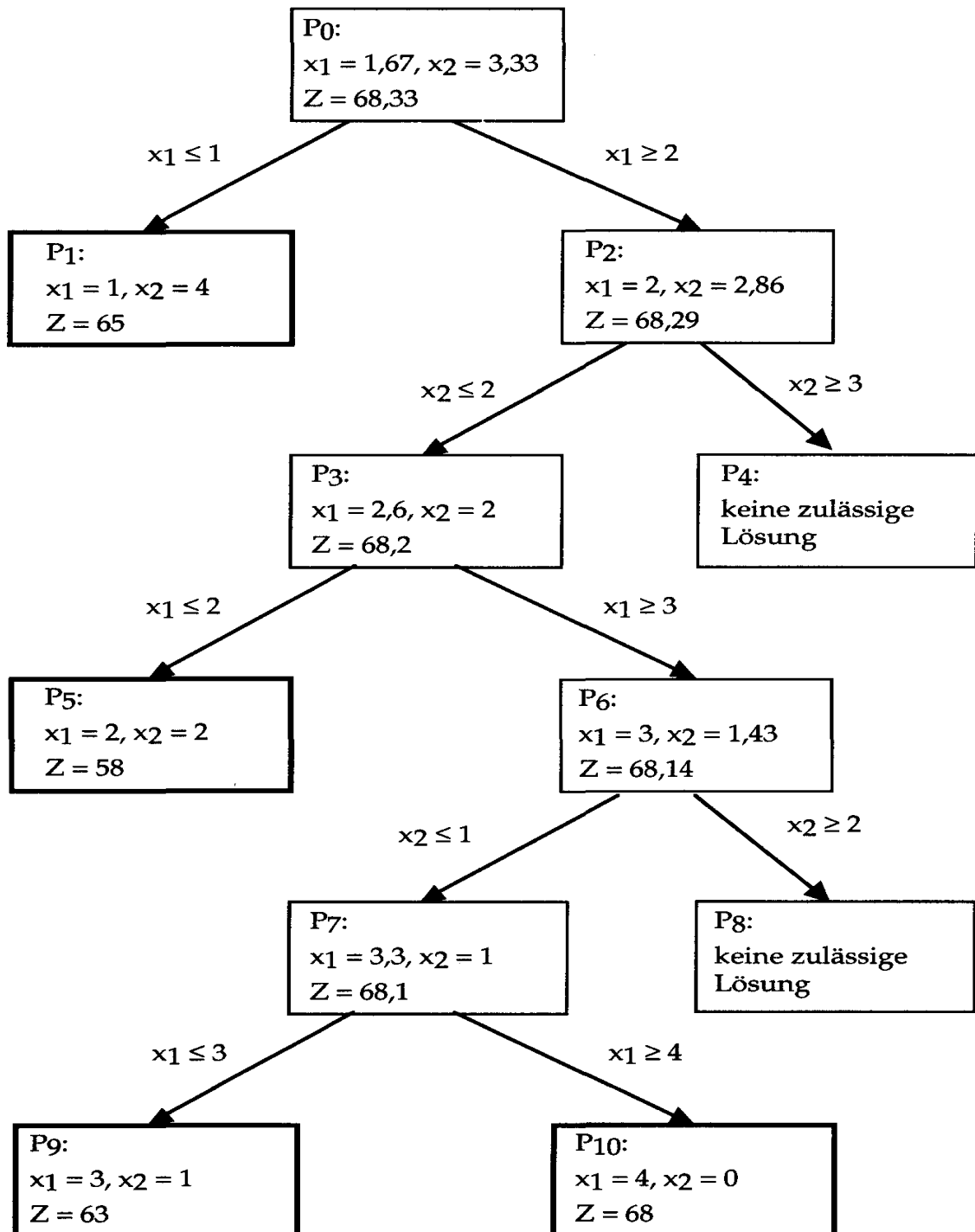
In allen drei Fällen braucht das Teilproblem nicht weiter untersucht zu werden. Man wählt daher ein anderes Teilproblem aus und wiederholt den Prozess bis man über alle Teilprobleme eine der drei obigen Informationszustände erreicht. Ist der Prozess abgeschlossen, so erhält man mit der besten bekannten zulässigen Lösung die Optimallösung des Gesamtproblems.

Explizit enumeriert werden dabei nur die ganzzahligen Lösungen der Teilprobleme, die sich im Laufe des Entscheidungsprozesses als zulässig herausgestellt haben. Alle anderen Lösungen werden von der jeweils besten bekannten Lösung dominiert und damit implizit berücksichtigt. Dies ist der Nachweis der Optimalität.

Im obigen Beispiel greift der Branch and Bound Algorithmus nicht. Hier können keine Teilprobleme ausgeschlossen werden und es erfolgt (worst case) die **vollständige Enumeration**.

Das Branch and Bound Verfahren wird noch einmal am Rucksackproblem thematisiert werden.

Es erfolgt zunächst (auf Seite 33) eine Berechnung mithilfe eines Schnittebenenverfahrens.

Enumerationsbaum

Schnittebenenverfahren

Erläuterung anhand des vorangegangenen Beispiels

Das obige Beispiel wird mit dem Simplex Algorithmus ohne Ganzzahligkeitsforderung gelöst. Es ergibt sich folgendes Endtableau:

X_1	X_2	Y_1	Y_2		
0	0	5/3	1/3		68 1/3
1	0	1/3	-2 1/3		1 2/3
0	1	-1/3	3 1/3		3 1/3

Das Lösungspolyeder wird nun beschnitten

Aufgaben zur ganzzahligen Programmierung

Aufgabe 1

Gegeben ist folgendes Lineares Programm:

$\max x_1 + 2x_2$ unter den Nebenbedingungen

$$3x_1 + 5x_2 \leq 15$$

$$5x_1 + 20x_2 \leq 46 \quad x_1, x_2 \geq 0 \text{ und ganzzahlig.}$$

Das LP wird zunächst ohne Ganzzahligkeitsforderung gelöst.

Das Endtableau des Simplex-Algorithmus lautet:

X_1	X_2	Y_1	Y_2	RS	
0	0	2/7	2/70	5.6	Zielfunktionszeile
1	0	4/7	-1/7	2	
0	1	-1/7	6/70	1.8	

Wenden Sie das Schnittebenenverfahren an:

- Geben Sie die beiden möglichen Restriktionen an.
- Bestimmen Sie die Restriktion die zur aktuellen (nicht ganzzahligen) Lösung den größten Abstand hat.
- Ergänzen Sie nun das Tableau um diese neue Restriktion und berechnen Sie das folgende Tableau.

Aufgabe 2

Gegeben ist folgendes Lineares Programm:

max $3x_1 + 4x_2$ unter

$$2x_1 + 3x_2 \leq 6$$

$$2x_1 + 1x_2 \leq 5$$

$x_1, x_2 \geq 0$ und ganzzahlig.

Das Endtableau des Simplex-Algorithmus lautet:

x_1	x_2	y_1	y_2	RS
0	0	5/4	1/4	35/4
0	1	1/2	-1/2	1/2
1	0	-1/4	3/4	9/4

Wenden Sie das Schnittebenenverfahren an:

- Bestimmen Sie die Restriktion die zur aktuellen (nicht ganzzahligen) Lösung den größten Abstand hat.
- Ergänzen Sie nun das Tableau um diese neue Restriktion und berechnen Sie die Schnittgerade in Abhängigkeit der Strukturvariablen.

Aufgabe 3

Gegeben ist folgendes LP:

Maximiere $2x_1 + 3x_2$ unter

$$x_2 \leq 4$$

$$5x_1 + 6x_2 \leq 30 ; x_1, x_2 \geq 0 \text{ und ganzzahlig.}$$

Das Anfangstableau des Simplex-Algorithmus lautet:

x_1	x_2	y_1	y_2		
-2	-3	0	0	0	Zielfunktionszeile
0	1	1	0	4	
5	6	0	1	30	

Das Endtableau des Simplex-Algorithmus lautet:

x_1	x_2	y_1	y_2		
0	0	0,6	0,4	14,4	Zielfunktionszeile
0	1	1	0	4	
1	0	-1,2	0,2	1,2	

Wenden Sie das Schnittebenenverfahren an:

- Ergänzen Sie das Tableau um die erste neue Restriktion.
- Geben Sie die dazugehörige Schnittgerade an.

Branch and Bound Verfahren, Heuristik und Dynamische Optimierung am Beispiel des Rucksackproblems**Problemstellung**

Aus einer Menge von n Objekten, die jeweils ein Gewicht und einen Nutzwert haben, soll eine Teilmenge ausgewählt werden, deren Gesamtgewicht eine vorgegebene Gewichtschränke K nicht überschreitet. Unter dieser Bedingung soll der Nutzwert der ausgewählten Objekte maximiert werden.

Anwendungsbeispiele hierfür sind:

Investitionen die entweder ganz oder gar nicht getätigt werden können. Jede Investition hat einen Nutzen. Das zur Verfügung stehende Budget sei K .

Beladen eines LKW: Jedes Paket hat ein Gewicht und eine Dringlichkeit (Nutzen). Das gesamte zulässige Gewicht sei K .

Packen eines Rucksacks für eine Wanderung ...

Mathematisches Modell

Gegeben seien n Objekte:

Sei w_i der Wert und g_i das Gewicht des i -ten Objekts mit $w_i, g_i \geq 0$ für $i = 1, \dots, n$.

Das LP lautet:

Zielfunktion: *maximiere* $W = \sum w_i x_i$

unter der Restriktion: $\sum g_i x_i \leq K$

$x_i \in \{0, 1\}$ für $i = 1, \dots, n$

Erläuterung: $x_i = 1$: das Objekt wird ausgewählt, $x_i = 0$: das Objekt wird nicht ausgewählt.

Beispielaufgabe

Gegeben sei ein Budget von 7.000 GE und 4 Projekte in die investiert werden können. Nicht verwendete „Budgetreste“ können nicht zinsbringend angelegt werden.

Projekt	Investition (in GE) (g_i)	Jahresgewinn (in GE) (w_i)
1	2000	9000
2	3000	13000
3	5000	20000
4	4000	14000

Ein **heuristisches Verfahren** ist ein Näherungsverfahren das in der Regel keine exakte Lösung ermittelt.

Hier ein mögliches heuristisches Verfahren:

Ordne die Objekte (hier die Investitionen) mit fallender „Rentabilität“ r_i ($r_i = w_i/g_i$) und teile die zur Verfügung stehenden Mittel der Reihe nach zu.

Heuristisches Verfahren (hier ein Greedy Algorithmus):

Projekt	Rentabilität = Jahresgewinn/Investition	Übrige Mittel
1	$9/2 = 4,5$	5
2	$13/3 = 4,33$	2
3	$20/5 = 4$	-3
4	$14/4 = 3,5$	-----

Nach diesem heuristischen Verfahren wird in die ersten beiden Projekte investiert.

Der Gewinn beträgt 22.000 GE. Es ist offensichtlich, dass dies nicht die Optimallösung ist.

Branch and Bound Verfahren

In jeder Stufe des Enumerationsbaumes werden folgende Werte notiert:

(Obere Schranke / Restkapazität / aktueller Nutzen).

(56 / 7 / 0)

$X_1 = 1$ (56 / 5 / 9)

$X_1 = 0$ (47 / 7 / 0)

...

Vervollständigen Sie den Baum. Suchen Sie zunächst eine erste zulässige Lösung, indem Sie sukzessive möglichst viele Variablen „eins“ setzen. Mit dem so erhaltenen Gesamtnutzen (der in der Regel noch nicht optimal ist) vergleichen Sie im weiteren Verlauf die oberen Schranken. Liegt der aktuell größte Gesamtnutzen über einer oberen Schranke, können Sie diesen Ast „abschneiden“. Erhält man im weiteren Verlauf der Rechnung einen höheren Gesamtnutzen als der aktuelle Wert, wird dieser ersetzt.

Dynamische Programmierung

Die Methode wurde ursprünglich für die Optimierung mehrerer aufeinander folgender Planungsperioden entwickelt. Die Zeitabhängigkeit der Problemstellung hat der Methode ihren Namen gegeben. Die Methode ist aber allgemeiner einsetzbar, die Bezeichnungen mehrstufige oder rekursive Optimierung wären treffender.

Lösungsweg für obiges Beispiel:

Um den maximalen Zielfunktionswert zu berechnen, wird in $n = 4$ Stufen optimiert.

Hierbei wird in der ersten Stufe nur das erste Objekt betrachtet, in der zweiten Stufe das erste und das zweite Objekt erfasst, in der dritten Stufe die ersten drei Objekte und, in diesem Beispiel zuletzt alle vier Objekte einbezogen.

Lösung:

$W_i (g_i, w_i)$	0	1	2	3	4	5	6	7
0 (0, 0)	0	0	0	0	0	0	0	0
1 (2, 9)	0	0	9	9	9	9	9	9
2 (3, 13)	0	0	9	13	13	22	22	22
3 (5, 20)	0	0	9	13	13	22	22	29
4 (4, 14)	0	0	9	13	14	22	23	29

Es werden die Projekte 1 und 3 durchgeführt

Keep	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1
2	0	0	0	1	1	1	1	1
3	0	0	0	0	0	0	0	1
4	0	0	0	0	1	0	1	0

Lösungsweg:

1. Erstellen von zwei Matrizen V und $Keep$.
2. Die ersten Zeilen und Spalten beider Matrizen werden mit Nullen ausgefüllt.
3. Man arbeitet sich Zeile für Zeile, und innerhalb der Zeile von links nach rechts, vor.
Für jede Zeile in der ersten Tabelle (W) wird abgefragt:

Passt das Projekt (g_i) in das Gesamtgewicht?

- a. Ja: Ist der Wert (w_i) addiert mit dem Wert des Restgewichts aus der Zeile davor größer, als der Wert der Zelle darüber?
 - i. Ja: Addierte Werte werden eingefügt
 - ii. Nein: Der Wert aus der Zelle darüber wird übernommen
 - b. Nein: Der Wert der Zelle darüber wird übernommen
4. Parallel dazu wird die $Keep$ -Matrix ausgefüllt.

Graphen-Algorithmen

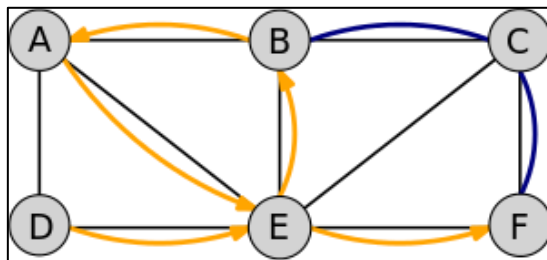
Definition

Ein Graph $G(V, E)$ besteht aus einer Menge von Knoten V (vertex) und Kanten E (edge).

Die Menge der Kanten E ist eine Teilmenge des kartesischen Produkts: $E \subseteq V \times V$.

Strecken verbinden die Punkte des Netzwerkes

Beispiel: Landkarte mit Städten als Knoten und Straßen als Kanten



Definitionen:

1. Gerichtete Kante:

- Eine Kante verläuft nur in eine Richtung (gelbe Kanten)
- Sind alle Strecken gerichtet, so spricht man von einem gerichteten Graph.

2. Kantenfolge:

- Eine endliche Folge von benachbarten Kanten
- Beispiel: Wegbeschreibung mit mehreren Straßen (Kanten)

3. Pfad:

- Kantenfolge, die nur gerichtete Kanten enthält.
- Beispiel: A-E-F

4. Kreis (Schleife oder Zyklus):

- Kantenfolge, die einen Knoten mit sich selbst verbindet
- Beispiel: A-E-B-A

5. Zusammenhängender Graph

- Jedes Knotenpaar ist durch mindestens eine Kantenfolge verbunden

6. Baum:

- Ein zusammenhängender, kreisfreier und ungerichteter Graph.
- Bei N Knoten gibt es $N-1$ Kanten.

7. Gewichteter Graph

- Jeder Kante des Graphen wird eine reelle Zahl zugeordnet.

8. Adjazenzmatrix

- Graphen können mithilfe von Matrizen dargestellt werden.
Für n Knoten wird in einer $n \times n$ Matrix (a_{ij}) die Anzahl der (gerichteten) Kanten von Knoten i nach Knoten j angegeben.

9. Adjazenzlisten

- Verkettete Listen bestehend aus allen Knoten, die mit dem betrachteten Knoten verknüpft sind. Der betrachtete Knoten bildet den Anker der Liste.

Minimaler Spannbaum:

Anwendung:

In einem Netzwerk gilt es alle Teilnehmer miteinander zu verbinden. Gesucht ist ein möglichst kostengünstiges Netz.

Mathematisch:

Gegeben ist ein zusammenhängender, gewichteter ungerichteter Graph mit n Knoten, m Kanten und den Gewichten c : $G(V, E, c)$. Gesucht ist der minimale Spannbaum dieses Graphen, das ist der kreisfreie, zusammenhängende Teilgraph $T(V, E', c)$ von G , bei dem die Summe der Kantengewichte minimal sind.

Wir betrachten hier den **Kruskal-Algorithmus**

1. Sortiere alle Kanten der Größe nach: $c(e_1) \leq c(e_2) \leq \dots c(e_i) \leq \dots c(e_m)$

2. Setze in $T(V, E')$ $E' = \{\}$

3. Führe m Iterationen aus: $i = 1 \dots m$

{

Wähle Kante e_i und prüfe, ob die Hinzunahme zu $T(V, E')$ keinen Kreis erzeugt.

Wenn ja: $E' = E' \cup e_i$

Abbruch des Verfahrens, wenn E' genau $n-1$ Kanten enthält

}

Kürzeste Wege in Graphen

Gesucht ist die Kantenfolge von einem Startknoten zu einem Zielknoten, für die die Summe der Kantengewichte minimal ist. Es werden Baumalgorithmen unterschieden, die die kürzeste Verbindung zwischen einem Startknoten S und allen übrigen Knoten bestimmen, und Algorithmen, die die kürzeste Verbindung zwischen allen Knotenpaaren liefern.

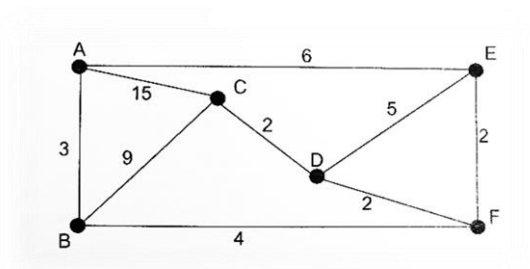
Zu den Baumalgorithmen gehört der **Dijkstra-Algorithmus**.

Voraussetzungen:

Gewichteter, gerichteter Graph, alle Gewichte sind nicht negativ. $G(V,E,c)$, n Knoten, $c(i,j)$ Gewicht der Kante von Knoten i nach Knoten j .

Beispiel: Gegeben ist folgendes Netz (Stadtplan):

Alle Kanten sind hier als gerichtet anzusehen. D.h., die Gewichte der Kanten gelten für beide Richtungen. Gesucht ist der kürzeste Weg von Startknoten A zu allen anderen Knoten. Geben Sie die kürzesten Wege an.



Initialisierung			
Knoten	Aktuelle Reisezeit zum Startknoten [min]	Vorgänger-Knoten	definitiv
A	0	-	X
B	∞		
C	∞		
D	∞		
E	∞		
F	∞		

Weitere Schritte			
Knoten	Aktuelle Reisezeit zum Startknoten [min]	Vorgänger-Knoten	definitiv
A	0	-	X
B	3	A	
C	15	A	
D	∞		
E	6	A	
F	∞		

Weitere Schritte			
Knoten	Aktuelle Reisezeit zum Startknoten [min]	Vorgänger-Knoten	definitiv
A	0	-	X
B	3	A	X
C	11	D	X
D	9	F	X
E	6	A	X
F	7	B	X

Notizen			
Keine unbesuchten Knoten von C aus. Dijkstra-Algorithmus ist fertig			

Dijkstra Algorithmus

Folgende Größen werden verwendet:

$D[1..n]$: $D[i]$ kürzeste Entfernung von S zum Knoten i .

$R[1..n]$: $R[i]$ unmittelbarer Vorgängerknoten von Knoten i auf dem kürzesten Weg von S nach i

Initialisierung :

$$MK = \{S\}$$

$$D[S] = 0$$

$$D[i] = \infty \text{ für } i \neq S$$

Iteration :

Wiederhole bis $MK = \{\}$

{

Wähle $h \in MK$ mit $D[h] = \min \{D[i], i \in MK\}$

Führe für alle unmittelbaren Nachbarknoten j von h aus

{

If $D[j] > D[h] + c(h, j)$ then

$$\{D[j] = D[h] + c(h, j)$$

$$R[j] = h$$

$$MK = MK \cup \{j\}$$

}

}

Entferne h aus MK

}

3. Simulation

Man unterscheidet grundsätzlich zwischen Computereperimenten und Monte Carlo Simulationen. Letzteres sind Verfahren zur Problemlösung unter Verwendung von Zufallszahlen.

Zufallszahlen

Definition

Zufallszahlen sind eine Folge von Zahlen, bei der keine Gesetzmäßigkeit erkennbar ist. Mithilfe eines Algorithmus können demnach nur Pseudozufallszahlen generiert werden. Dies sind Folgen von Zufallszahlen, die mehrmals durchlaufen werden, die also periodisch sind. Ist die ursprüngliche Folge lang genug, spielt die Wiederholung keine Rolle.

Ein bekanntes Verfahren ist die multiplikative Kongruenz nach Lehmer:

$x_{n+1} = (a \cdot x_n + c) \bmod m$ mit $0 \leq m$, $0 < a < m$, $0 \leq c < m$, $0 < m$, alle Zahlen sind natürliche Zahlen.

x_0 nennt man Saat, a den Multiplikator, c Inkrement und m Modul.

Bestimmen Sie die Zufallszahlenfolge für

- 1) $a = 4$, $m = 9$, $c = 2$ und dem Startwert 1,
- 2) $a = 6$, $m = 9$, $c = 2$ und dem Startwert 4.

Ein guter Zufallsgenerator sollte gleichmäßig verteilte Zahlen erzeugen.

Ein eher schlechtes Beispiel:

- 3) $a = 81$, $m = 200$, $c = 41$ und dem Startwert 79.

Weitere Auswahlkriterien für die Faktoren a , c und m sind nach Prof. D. Knuth definiert worden.

m ist weiterhin dem Zahlbereich des Rechners angepasst. Für einen 32-Bit Rechner wird $m = 2^{32} - 1$ gewählt und es ergibt sich dann für alle Startwerte eine Periode von circa 2^{19} .

Monte Carlo Verfahren

Monte Carlo Simulationen sind Verfahren, bei denen eine geschickte Verwendung von Zufallszahlen die näherungsweise Lösung eines Problems liefert.

Ein einfaches Beispiel für diese Grundidee ist die Berechnung der Zahl Pi.

Ein weiteres Beispiel stellt die Monte Carlo Integration dar.

Gesucht ist der Wert des Integrals $I = \int_a^b f(x)dx$.

Dieses Integral kann durch die Formel $I = \int_a^b f(x)dx \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i)$ approximiert werden. Hierbei sind alle $x_i \in [a, b]$ generierte Zufallszahlen. Gerade für Funktionen die keine Stammfunktion besitzen, ist das ein praktikables und schnelles Verfahren.

Das Verfahren kann beispielsweise zur Berechnung der Wahrscheinlichkeiten der Gaußschen Normalverteilung herangezogen werden.

Übung:

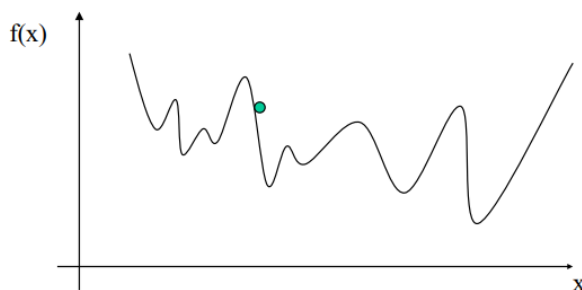
Berechnen Sie das Integral $I = \int_0^8 x^2 dx$ mit der ersten Zahlenfolge von oben. Wie weit sind Sie von der exakten Lösung entfernt?.

Simulated Annealing

Das bekannteste Monte Carlo Optimierungsverfahren heißt Simulated Annealing. Der Begriff ist aus einem physikalischen Verfahren übernommen. Bei der Herstellung kristalliner Strukturen durch Abkühlen aus der Schmelze wird zwischenzeitlich immer wieder kurz erhitzt. Dadurch erzielt man besonders reine Kristalle.

Auf der Abstraktionsebene mathematischer Optimierungsprobleme lässt sich das Verfahren auf betriebswirtschaftliche Fragen übertragen.

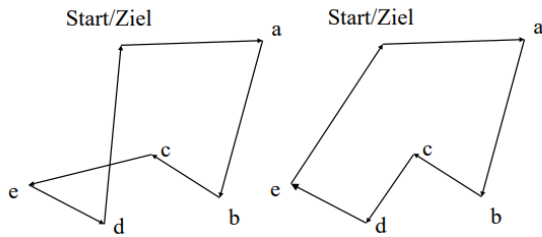
Gesucht wird beispielsweise das globale Minimum (Maximum) einer Funktion



Würde man bei der Suche an einer bestimmten Stelle beginnen und dann eine schrittweise Verbesserung der Lösung anstreben, so würde man nur das erste lokale Minimum berechnen. Simulated Annealing ist ein Algorithmus, der in der Lage ist, kontrolliert auch schlechtere Varianten zu akzeptieren. Die Wahrscheinlichkeit eine schlechtere Lösung zu akzeptieren, wird aber nach und nach verringert. Wäre dem nicht so, würde die Suche nach einer Lösung einem ziellosen Irren im Lösungsraum gleichen.

Simulated Annealing ist somit ein probabilistischer Algorithmus zur Approximation des globalen Optimums einer Funktion.

Ein weiteres Beispiel für die Anwendung von Simulated Annealing ist das Travelling Salesman Problem: Hierbei geht es darum in einer Rundreise alle vorgegebenen Orte genau einmal aufzusuchen, zum Ausgangsort zurückzukehren und dabei eine möglichst kurze Distanz zurückzulegen.



Anfangs würde der Algorithmus zufällige Routen ausprobieren und diese basierend auf ihrer Länge bewerten. In den ersten Iterationen würden auch längere Routen akzeptiert werden um eine breite Suche zu ermöglichen. Später wird die Wahrscheinlichkeit, dass schlechtere Routen akzeptiert werden nach und nach verringert. Durch mehrere Iterationen findet der Algorithmus schließlich eine sehr gute Lösung, die unter Umständen sogar die Optimallösung sein kann.

Ablauf des Algorithmus

1. Starte mit einer zufälligen Lösung.
2. Generiere eine neue, zufällige Lösung in der Nachbarschaft der aktuellen.
3. Bewertung: Ist die neue Lösung besser, so wird sie übernommen. Ist sie schlechter, so wird sie mit einer gewissen Wahrscheinlichkeit akzeptiert.
4. Wiederhole die Schritte 2) und 3) mehrere Male. Im Laufe der Wiederholungen der Schritte wird die Wahrscheinlichkeit, schlechtere Lösungen zu akzeptieren, verringert.
5. Wiederhole die Schritte 2) bis 4), bis ein vorgegebener Mindestwert erreicht wird oder bis eine weitere Verbesserung der Lösungen ausbleibt.

Mit der Definition einer Wahrscheinlichkeitsfunktion (punkt 3) ergibt sich hieraus:

$\vec{x}_a$: *aktuelle Lösung*

$\vec{x}_n$: *neue Lösung*

$z = f(\vec{x})$: *Zielfunktion*

$\Delta z = f(\vec{x}_n) - f(\vec{x}_a)$

β : *Parameter* > 0

$$p(\Delta z, \beta) = \begin{cases} 1 & \text{für } \Delta z \leq 0 \\ \exp(-\beta \cdot \Delta z) & \text{für } \Delta z > 0 \end{cases}$$

1. Wähle beliebige Startlösung $\vec{x}_a$
2. Wähle für β eine kleine positive Zahl
3. Verändere die aktuelle Lösung $\vec{x}_a$ geringfügig $\rightarrow \vec{x}_n$
4. Berechne Δz und $p(\Delta z, \beta)$
5. Wähle Zufallszahl $RN \in [0,1]$
6. Für $RN \leq p(\Delta z, \beta)$ $\vec{x}_a = \vec{x}_n$
 Sonst Lösche $\vec{x}_n$
7. Gehe (n) mal zu 3.
8. Erhöhe β
9. Gehe (m) mal zu 3.